

Getting Started with Analysis of Boolean Functions

Arnav Bhargava

August 2026

1 Preliminaries

1.1 Boolean Functions

Definition 1.1 (Boolean Function). A boolean function $f(x) = f(x_1, \dots, x_n)$ of n variables is a mapping $f : \{0, 1\}^n \rightarrow \{0, 1\}$. It is *symmetric* if the order of inputs does not matter i.e. only depends on the number of 1s in the input vector.

$$\exists \phi : [n] \rightarrow \{0, 1\}, f(x_1, \dots, x_n) = \phi(x_1 + x_2 + \dots + x_n)$$

- Threshold functions $Th_k^n(x) = 1. \iff x_1 + \dots + x_n \geq k$
- Majority functions $Maj_n(x) = 1. \iff x_1 + \dots + x_n \geq \lceil n/2 \rceil$
- Parity functions $\bigoplus_n(x) = 1 \iff x_1 + \dots + x_n \equiv 1 \pmod{2}$
- Modular functions $MOD_k = 1 \iff x_1 + \dots + x_n \equiv 0 \pmod{k}$

Any property can be expressed as a boolean function.

Example 1.1 (Binary cube). The set $\{0, 1\}^n$ of all binary vectors is called the binary n -cube. The subcube of dimension d Binary n -cube is considered as a graph Q_n , vertices are all 2^n bit strings of n bits. Edges are in between any two vertices that differ in one bit in their label. Every boolean function is $f : \{0, 1\}^n \rightarrow \{0, 1\}$ is just a coloring of vertices in Q_n in two colors. Let the bipartite subgraph G_f of Q_n , be obtained by removing all edges joining the vertices in the same color class. If d_a denotes the average degree in G_f of vertices in the color class $f^{-1}(x)$, $a = 0, 1$ then the product $d_0 \cdot d_1$ is a lower bound on the length of any formula computing f using $\wedge, \vee, \neg$.

Definition 1.2 (CNFs and DNFs). One way to represent a boolean function is to give the entire truth table. Or we can give more compact representations obtained by looking at $f^{-1}(0)$ or $f^{-1}(1)$.

- *literal* is a boolean variable or it's negation.

- *consistent monomial* is an AND of literals and doesn't contain x_i and $\overline{x_i}$ at the same time.
- *consistent clause* is an OR of literals and doesn't contain x_i and $\overline{x_i}$ at the same time.

A CNF is an AND of clauses. A DNF is a OR of monomials. Every boolean function $f(x)$ of n variables can be written both a DNF $D(x)$ and a CNF $C(x)$:

$$D(x) = \bigvee_{a: f(a)=1} \bigwedge_{i=1}^n x_i^{a_i}$$

$$C(x) = \bigwedge_{b: f(b)=0} \bigvee_{i=1}^n x_i^{1-b_i}$$

The *width* of a CNF or DNF formula is the maximum number of literals in monomial or clause.

Example 1.2. The set of vectors accepted by monomial $\overline{x_1} \wedge x_3$ is a subcube of dimension $n - 2$ and represents $a = (0, *, 1, \dots, *)$. The set of vectors rejected by clause $\overline{x_1} \vee x_3$ is a subcube of dimension $n - 2$ and represents $a = (0, *, 1, \dots, *)$.

Definition 1.3. For two vectors $x, y \in \{0, 1\}^n$ we write $x \leq y$ if $x_i \leq y_i$ for all position i . A Boolean function is *monotone*, if $x \leq y \implies f(x) \leq f(y)$.

$Th_k^n(x)$, $Clique(n, k)$, AND , OR are all monotone. An *implicate* of a boolean function $f(x_1, \dots, x_n)$ is a **clause** such that c such that $f \leq c$, which is the same as as saying $\forall a \in \{0, 1\}^n, c(a) = 0 \implies f(a) = 0$. An *implicant* of a boolean function $f(x_1, \dots, x_n)$ is a **term** such that t such that $t \leq f$, which is the same as as saying $\forall a \in \{0, 1\}^n, t(a) = 1 \implies f(a) = 1$. An implicant/implicate is prime if no proper subclause is also an implicant/implicate.

Proposition 1.1. Prime implicants of monotone boolean functions have no negated variables.

Proof. Suppose $t = (\wedge_S x_i) \wedge (\wedge_T \overline{x_i})$ is a term that is a prime implicant. Let $t^+ = (\wedge_S x_i)$. There is a vector $a \in \{0, 1\}^n$ which is $a_i = 1, \forall i \in S$ that makes just $t^+(a) = 1$ but $f(a) = 0$ (since t is a prime implicant this must be true). Now let b be a vector such that $\forall j \in T, i \in S, b_i = a_i, b_j = 0$ which makes $t(b) = 1$. As $b \leq a$ and f is monotone then we have that $f(b) = f(a) = 0$. But this is a contradiction of t being an implicant, $t(b) = 1$ but $f(b) \neq 1$. $\square$

Definition 1.4. A 1-term of a boolean function is a subset of it's variables such that after setting them to 1 or 0 the function becomes a constant 1. Minterms are 1-terms which are minimal under set-theoretic inclusion.

Proposition 1.2. If all minterms of f are of length at most k , then f can be written as a k -DNF; that is, as an OR of terms, each involving at most k literals.

Proof. By definition, each minterm corresponds to a term (an AND of literals) that implies $f = 1$, and minimality ensures no proper subset of its literals suffices. If every minterm has length at most k , then the DNF formed by taking the OR over all minterms yields a k -DNF representation of f . $\square$

Definition 1.5 (m-representable). f is m -representable if there is an $m \times n$ matrix $A = (a_{ij})$ with nonnegative real entries, and a nonnegative vector b such that $f(x) = 0$ iff $Ax \leq b$. Let $m(f)$ denote the smallest number m (of rows) such that f is m -representable.

Definition 1.6 (Hamming Function). Any monotone boolean function is a Hamming Function if any two distinct lowest ones differ in more than 2 positions. It is homogenous if for all vectors its lowest ones have the same number of 1-positions.

Theorem 1.3 (Zuev and Trishin, 1983). *Let f be a monotone homogenous boolean function. If f is also a Hamming function then $m(f) \geq |L_f|$.*

$f = \text{CLIQUE}(n, k)$ is a Hamming function, every two distinct k -cliques in K_n must differ in $\geq 2(k-1) \geq 4$ edges.

Here are two adhoc lemmas that are quite useful.

Let G be a DAG with one source node (the root) from which all leaves (nodes of outdegree 0) are reachable. Suppose each non-leaf node has outdegree $\leq k$. Each vertex is assigned a non-negative weight which is *subadditive*: the weight of the node does not exceed the sum of the weights of its successors. Let r be the weight of the root, and leaves have weight $l < r$.

Lemma 1.4 (Balanced Subgraphs). For every real number ϵ between l/r and 1, there exists a node whose weight lies between $\epsilon r/k$ and ϵr . Every binary tree with r leaves has a subtree whose number of leaves lies between $r/3$ and $2r/3$.

The second claim comes from the fact that setting root weight r leaves now have weight 1. Take $\epsilon = 2/3$.

The *length* of a path will mean the number of nodes in it. The *depth* of a graph is the length of the longest path in it.

Lemma 1.5 (Valiant, 1977). Let $G = (V, E)$ be a DAG of depth $d = 2^k$. Then for every integer $1 \leq r \leq k$, it is possible to reduce the depth to $\leq d/2^r$ by removing $\leq \frac{r}{k}|E|$ edges.

Proof. First off every DAG has a canonical labelling i.e. a topological ordering. Let G be given a canonical labelling of its vertices, wherein each vertex gets a labelling equal to its eccentricity (longest path from any vertex to it). Let E_i be the set of all edges such that the endpoints differ in the i th bit (from the left) for the first time. There are $k = \log_2 d$ such sets. These sets are disjoint. To verify: w.l.o.g let $i < j$, then for $e \in E_i \cap E_j$, it must have already differed in i before j , which is a contradiction. Moreover, the union of E_i s is equal to E .

Now let E_i is removed from the graph, then we can relabel the nodes using integers $1, \dots, d/2 = 2^{k-1}$, by deleting the i th bit in the binary representations of labels. The new labeling l' is legal since if (u, v) survives then the first difference of the old labels $l(u) < l(v)$ was on some $j \neq i$. If $j > i$ and thus $l(v) > 2^j, l(u) < 2^j$, then deleting i ensures $l'(u) < l'(v)$. If $j < i$ then the binary representation is the same up until j , so removing i still preserves $l'(u) < l'(v)$. $\square$

1.2 Circuits

Here are some definitions:

Definition 1.7 (n variable Circuit or SLP over basis $\mathcal{B}$). Let $\mathcal{B}$ be a set of Boolean functions. A circuit of n variables over the basis $\mathcal{B}$ is a sequence $g_1, \dots, g_t$ of $t \geq n$ boolean functions (called gates). Moreover, each gate $g_i = \phi(g_{i_1} \dots g_{i_d})$. Where each g_{i_j} is either one of the input variables, or some previous gates.

A basis $\mathcal{B}$ is complete if every Boolean function can be computed by a circuit over this basis.

Definition 1.8 (Fan-in of a Circuit). Fan-in is the maximum number of inputs a given gate present in the circuit takes.

A circuit is equivalent to a DAG whose 0-fan-in nodes correspond to variables, each other node v corresponds to a function in $\mathcal{B}$. Finally, one or more are distinguished as outputs.

Definition 1.9 (Size). Size of the circuit is the total number of its gates.

Definition 1.10 (Depth). Depth of the circuit is the maximum number on a path from an input variable to an output gate.

We consider the classical model where the gates are from the set $\{\wedge, \vee, \neg\}$, where $\{\wedge, \vee\}$ is of Fan-in 2. It is universal meaning we can compute any boolean function using it. The monotone basis $\{\wedge, \vee\}$ (excluding NOT) is NOT universal.

Example 1.3. The binary basis B_2 consist of all $2^{2^2} = 16$ possible functions on 2 variables. These functions prescribe a basis circuits.

A *De Morgan circuit* is a circuit with fanin-2 gates over the basis $\{\wedge, \vee\}$ whose inputs are the variables $x_1, \dots, x_n$ and their negations $\bar{x}_1, \dots, \bar{x}_n$. We push all negations to the top of the circuit (we can view them as literals) such that the remainder of the circuit is monotone.

Proposition 1.6. For any circuit of size n , all negations can be moved to the inputs (i.e. can be made into a De Morgan circuit) by increasing the number of gates at $\leq 2n$.

If we want to take the NOT of some output gate (output of AND or OR) then we can use DeMorgan rules $\neg(x \vee y) = \neg x \wedge \neg y$ and $\neg(x \wedge y) = \neg x \vee \neg y$ to substitute them with the output of literals instead.

Proposition 1.7. Every symmetric Boolean function $f_n : \{0, 1\}^n \rightarrow \{0, 1\}$ can be computed by a circuit with basis $\{\wedge, \vee, \neg\}$ of size $\leq 9n + o(n)$.

A full adder circuit $FA_3(x, y, z) = (c, s) \in \{0, 1\}^2$ of three variables computes the binary representation of the (real) sum $x + y + z \in \{0, 1, 2, 3\}$.

1.2.1 Definition of P/Poly

The language L over $\{0, 1\}$ accepted by a Turing Machine (TM) is a subset of bit-strings from $\{0, 1\}^*$ – set of all finite bit-strings. In the world of TMs, if a language L is in **P**, then some TM M can 'run on' strings $x \in \{0, 1\}^*$ and decide whether $x \in L$ or not in polynomial time. TMs are a *uniform* model of computation meaning one fixed machine works for all input lengths. Can we define an equivalent complexity class $\mathcal{C}$ on circuits? The answer is yes, but we must be careful in translating between two notions. Namely, we must redefine:

1. our notion of running time
2. when L is in $\mathcal{C}$.

For the first, a circuit has no notion of 'polynomial running time' so instead we look at polynomial size bounded circuits. For the second, a circuit C over n variables computes a function $f(x_1, \dots, x_n) : \{0, 1\}^n \rightarrow \{0, 1\}$. However, our class $\mathcal{C}$ would be quite useless if we could only capture running on inputs of size n moreover, it would be significantly weaker than **P**. Instead, we focus on *circuit families* F which is an infinite list of circuits $F = \{C_0, C_1, \dots\}$ where C_n has n inputs. Define $\text{size}(F, n) = \text{size}(C_n)$. A circuit family F decides a language L if:

$$w \in L \iff C_k(w) = 1, |w| = k$$

We define our new complexity class $\mathcal{C}$ as:

$$\mathcal{C} := \{L \subseteq \{0, 1\}^* : \exists \text{ a circuit family } F \text{ deciding } L, \text{size}(F, n) \leq \text{poly}(n)\}$$

$\mathcal{C}$ is known as **P/Poly**. Since we require a different circuit for each input length, we work with circuit families not circuits themselves. This is what makes circuits a *non-uniform* model of computation.

1.2.2 Advice Definition of P/Poly

There is an alternative way to view **P/Poly**, which makes its non-uniform nature more explicit. Instead of circuits, we use Turing machines with *advice*.

A *polynomial advice function* is a mapping

$$a : \mathbb{N} \rightarrow \{0, 1\}^*$$

such that $|a(n)| \leq \text{poly}(n)$ for all n . Intuitively, $a(n)$ gives the machine extra information that depends only on the input length n , not on the particular input string.

A language $L \subseteq \{0, 1\}^*$ is in **P/Poly** if there exists a polynomial-time Turing machine M and a polynomial advice function a such that for every input $x \in \{0, 1\}^n$,

$$x \in L \iff M(x, a(n)) = 1.$$

That is, M runs in time polynomial in $|x|$ and has access to an extra advice string whose length is polynomially bounded.

Equivalence. The circuit-family definition and the advice definition are equivalent:

- Given a circuit family F , we can encode each $C_n \in F$ into an advice string $a(n)$, and let the Turing machine M simulate C_n on input x .
- Conversely, given M and $a(n)$, we can “hardwire” the advice into a polynomial-size circuit that simulates M on inputs of length n .

Why this matters. The advice definition highlights why **P/Poly** is more powerful than **P**. Since the advice string can depend arbitrarily on the input length n , it can encode information that no efficient computation could discover on its own. For example, if $A \subseteq \mathbb{N}$ is any set of natural numbers, we can build a unary language

$$L_A = \{1^n : n \in A\}$$

that lies in **P/Poly** by simply letting the advice $a(n)$ contain one bit indicating whether $n \in A$.

This shows two things:

- **P/Poly** strictly contains **P**.
- **P/Poly** even contains undecidable languages in the unary setting (since the advice can “bake in” non-computable information).

The circuit definition hides this phenomenon; the advice viewpoint makes it immediate. It also explains why **P/Poly** is called a *non-uniform* class: the extra information may vary with input length without any requirement that it be generated by a single uniform process.

1.2.3 Probabilistic Circuits

These circuits besides standard input variables $x_1, \dots, x_n$, some additional inputs $r_1, \dots, r_m$ called random inputs. These random bits are 2 from a uniform distribution from $\{0, 1\}$. The output of such a circuit $C(x)$ is a random 0-1.

Definition 1.11. $C(x)$ computes a Boolean function $f(x)$ if

$$\forall x \in \{0, 1\}^n, \Pr(C(x) = f(x)) \geq \frac{3}{4}$$

Lemma 1.8. (Majority trick) If $x_1, \dots, x_m$ are independent Bernoulli random variables with success probability $1/2 + \epsilon$ for $0 < \epsilon < 1/2$, then:

$$\Pr(\text{Maj}(x_1, \dots, x_m) = 0) \leq e^{-2\epsilon^2 m}$$

So this means that we can boost the probability of probabilistic circuits quite easily. They don't give much of an advantage compared to deterministic circuits. Namely, they are still part of P/poly.

Theorem 1.9 (Adleman, 1978). *Probabilistic polynomial-size circuit families collapse to deterministic polynomial-size circuit families with advice, i.e.*

$$\text{BPP/poly} = \text{P/poly}.$$

Proof sketch. Let $\{C_n\}$ be a family of probabilistic circuits of size $\text{poly}(n)$. For each input $x \in \{0, 1\}^n$, the circuit $C_n(x, r)$ takes as additional input a random string $r \in \{0, 1\}^{m(n)}$ and satisfies

$$\Pr_r[C_n(x, r) \neq f(x)] \leq \frac{1}{4}.$$

Error reduction. By independent repetition and taking the majority output, we can construct a new probabilistic circuit C'_n such that for all x

$$\Pr_r[C'_n(x, r) \neq f(x)] \leq 2^{-n}.$$

This follows from the Chernoff bound and only increases circuit size polynomially.

A union bound. For each fixed n , there are 2^n possible inputs. Since each input has error at most 2^{-n} , the probability (over random choice of r) that there exists some input x on which $C'_n(x, r)$ errs is at most

$$2^n \cdot 2^{-n} = 1.$$

Thus there exists a particular choice of random string r^* such that $C'_n(x, r^*) = f(x)$ for all $x \in \{0, 1\}^n$.

Hardwiring in the randomness. Define a deterministic circuit $D_n(x) := C'_n(x, r^*)$. The advice string for input length n is precisely the “good” random string r^* . Each D_n has size $\text{poly}(n)$ since C'_n did.

Therefore the family $\{D_n\}$ witnesses that $f \in \text{P/poly}$. $\square$

1.3 Networks and Branching Programs

Other models of computation include.

- Switching-and-Rectifier Networks (s.r.n) is a directed (not necessarily acyclic) with a source s and a target t . Edges can be *switches* if they are labelled with the truth value of a literal ($x_i = 1, \bar{x}_i = 0$). Unlabeled edges are *rectifiers*. Parallel edges are allowed.

- Contact schemes or undirected switching networks (underlying graph is undirected so only switches are allowed).
- Rectifier Networks: These are just SRNs without switches (only rectifiers), so they compute monotone functions.

The size of a s.r.n is the total number of labeled switches - rectifiers are essentially free. A s.r.n computes a function f if there is a valid directed path from s to t setting the switches to those supplied by the input.

1.3.1 Branching Programs

A *nondeterministic branching program* is an s.r.n whose underlying (directed) graph is acyclic. The complexity measure in this case is:

$$\text{NBP}(f) := \min_{G \in \mathcal{G}, G \text{ comp. } f} \text{size}(G)$$

$\text{NBP}_+(f)$ denotes the complexity of monotone n.b.p defined in the obvious way. A branching program is a log-space computations. Finding a computing a function via n.b.p $\equiv$ finding a consistent path in a $\text{NBP} \equiv_{\text{logspace}} \text{STCON}$ is NL-complete.

Theorem 1.10 (Immerman-Szelepcsényi). *For every sequence $(f_n), n \in \mathbb{N}$ of n -variate Boolean functions,*

$$\text{NBP}(\neg f_n) \leq \text{NBP}(f_n)^{O(1)}$$

Theorem 1.11 (Moore & Shannon, 1956 and Markov 1962). *For every monotone Boolean function f , $\text{NBP}_+(f) \geq l(f) \cdot w(f)$*

$l(f)$ – minimum length of min-term, $w(f)$ – minimum length of max-term.

2 Hard Functions

Given a circuit computing Boolean function f , together with $\mu(f)$ measuring the smallest possible number of gates necessary to compute f in this model. The *Shannon function* for this model is

$$\mu(n) = \max_{f(x_1, \dots, x_n)} \mu(f)$$

where the maximum is taken over all Boolean functions f of n variables.

For the class of circuits over the classical basis $\{\wedge, \vee, \neg\}$, we denote the $C(f) := \mu(f)$ and the respective Shannon function as $C(n) := \mu(n)$.

Lemma 2.1. There are at most $(s + 2n + 2)^{2s}$ DeMorgan circuits of size s . If $s \geq n + 2$, then upper bound is $2^{2s \log(2s)}$.

Proof Sketch. For each gate the input can either be $\wedge, \vee$ (2 choices), at most s other gates that are precomputed or literals ($2n$ choices). This is for its first input. For both inputs, the total number of choices are $(s + 2n + 2)^2$. Overall, for all s gates we have $(s + 2n + 2)^{2s}$ choices. If $s \geq n + 2$, then $(s + 2n + 2)^{2s} \approx 4s^{2s} = 2^{2s \log(4s)} = 2^{2s(\log(2s) + O(1))}$. DeMorgan circuits exist. $\square$

The proof above already applies a pretty loose upper-bound, even then, we can readily see that this is already fewer than the number of distinct Boolean functions over n variables.

Lemma 2.2 (Kannan, 1981). Let n be sufficiently large. For every integer $k \geq 1$ there is a Boolean function of n variables such that f can be written as a DNF with n^{2k+1} monomials, but $C(f) > n^k$.

Proof. A boolean function can be viewed as selecting a subset $f^{-1}(1) \subseteq \{0, 1\}^n$, and rejecting remaining vectors. Fix a subset $T \subseteq \{0, 1\}^n$ of size $|T| = n^{2k+1}$. By Lemma 2.1 we know at most $2^{n^{2k}} < 2^{|T|}$ distinct subsets can be accepted by circuits of size at most n^k . Thus there is still some subset $S \subseteq T$ cannot be accepted by a circuit of size n^k . But this subset S can be accepted by a trivial DNF with $|S| \leq |T| = n^{2k+1}$ monomials. $\square$

Theorem 2.3 (Shannon 1949; Muller, 1956). $C(n) = \Theta(2^n/n)$.

Comment on lower bound: To show $C(n) = \Omega(2^n/n)$. Setting $s := 2^n/10n$ we have by lemma 2.1 that $(2^n/5n) \log(2^n/5n) \approx (2^n/5n) \cdot n \approx 2^n/5$, and thus $2^{2^n/5} \ll 2^{2^n}$ which is much smaller than the number of distinct Boolean functions.

Proof Sketch. The upper bound is more involved.

Suppose we have a n variable function F . We now split the computation of F recursively using:

$$F(x_1, \dots, x_n) = (\bar{x}_n \wedge F_0) \vee (x_n \wedge F_1)$$

Here, F_0 and F_1 are the restrictions of F to $x_n = 0$ and $x_n = 1$, respectively; that is, $F_0 = F(x_1, \dots, x_{n-1}, 0)$ and $F_1 = F(x_1, \dots, x_{n-1}, 1)$.

The key here is we can stop doing this recursive computation at some point where feasible and compute the rest 'at once' in a single circuit. As in Figure 1. If we stop at a point F_{n-m} then we obtain a circuit computing F of size $3 \cdot 2^{n-m}$ (splitting introduces 3 gates at each step). This is essentially a big OR over all the possible $n - m$ sized bit-vectors appearing as the first $n - m$ inputs to f . The inputs to F_{n-m} are subfunctions on m variables (call it the set $\mathcal{H}_m$) and there are 2^{2^m} of them. Note that, for each $h \in \mathcal{H}_m$ there are 2^{n-m} circuits that will rely on the computation done by subfunction h .

The idea, therefore, is to stop at a point where $2^{n-m} \lesssim 2^{2^m}$.

The total circuit size (that computing F) can be decomposed into two contributions:

$$\underbrace{3 \cdot 2^{2^m}}_{\text{precompute all } m\text{-variable subfunctions}} + \underbrace{2^{n-m}}_{\text{wiring them into the recursion tree}}.$$

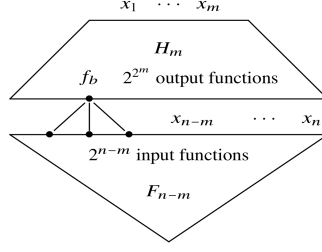


Figure 1: Recursive decomposition of F by fixing the last m variables.

Choosing $m \approx \log_2(n - \log_2 n)$ balances these two terms, yielding an overall circuit size of $\Theta\left(\frac{2^n}{n}\right)$. □

Theorem 2.4 (Lupanov 1958a, 1963).

$$(1 + \beta_n) \frac{2^n}{n} \leq C(n) \leq (1 + \alpha_n) \frac{2^n}{n}$$

where,

$$\alpha_n = O\left(\frac{\log n}{n}\right), \beta_n = (1 - o(1)) \frac{\log n}{n}$$

Proof Sketch. This is a clever construction of sub-circuits that together compute any function on n variables.

Assume $\text{bin}(a)$ is the integer represented by the binary vector a . The inputs to a function on n variables is a n bit vector representing an integer uniquely up to 2^n . Imagine splitting up this interval into m equal parts. On this interval define, $H_{n,m}(i)$ as the set of all Boolean functions $h(x)$ of n variables such that $h(a) = 0$ if $\text{bin}(a) \notin (m(i-1), mi]$, that is, $h(x)$ is zero outside of the i th interval out of the possible $2^n/m$. Now,

$$f(x) = \bigvee_{i=1}^{2^n/m} f_i(x)$$

where $f_i \in H_{n,m}(i)$ and $f_i(a) = f(a)$ when $\text{bin}(a) \in (m(i-1), mi)$. For each $a \in \{0, 1\}^n$ we can associate an elementary conjunction: $K_a = x_1^{a_1} \wedge x_2^{a_2} \wedge \dots \wedge x_n^{a_n}$.

Given this, and some well-chosen k , we can write $f(x)$ as follows:

$$f(x) = \bigvee_a K_a(x_1, \dots, x_k) \wedge \bigvee_{i=1}^{2^n/m} f_{a,i}(x_{k+1}, \dots, x_n)$$

The key observation is that all elementary conjunctions of n variables, that is all K_a , can be simultaneously computed in at most $2^n + 2n2^{n/2}$ gates. This is shown by

computing the conjunctions over the first $n/2$ variables using $(n/2) \cdot 2^{n/2}$ gates – $K_{[a]_{n/2}}$ has $n/2$ literals and there are $2^{n/2}$ assignments of them. Do this for both halves. Now combine by taking an AND of the corresponding outputs of these two circuits, requiring $2^{n/2} \cdot 2^{n/2} = 2^n$ additional gates.

We then construct a circuit F requiring 5 other sub-circuits, that together compute the function. (This is described in detail in the text).

The total size of such a circuit is:

$$\text{size}(F) \leq \frac{2}{m} + 4 \cdot 2^k + 2^{n-k} + 2n2^{k/2} + 2n2^{n-k} + 2^{n-k+m+1}$$

Set $k = n - 2 \cdot \log(n)$ and $m = n - 4 \cdot \log(n)$, such that all terms after the first are $O(2^n/n^2)$ and thus,

$$\text{size}(F) \leq \frac{2}{m} + O(2^n/n^2)$$

Simple manipulations then lead to the result in the form above. But the crux of the argument is encapsulated here. □

The above result gives credence to Shannon's bound; namely by showing that it is tight up to a constant factor.

2.1 Circuit Hierarchy Theorem

Like the time and space hierarchy theorems, this theorem essentially captures the idea that a 'little bit' more circuit size allows us to *strictly* increase the number of functions we can compute. The formal statement is as follows. Define a function $t : \mathbb{N} \rightarrow \mathbb{N}$, let $\text{Circuit}[t]$ denote the set of all sequences $(f_n)_{n \in \mathbb{N}}$ of Boolean functions on n variables such that $C(f_n) \leq t(n)$. That is $\text{Circuit}[t]$ is the set of all circuit families with size on all input lengths bounded by t .

Theorem 2.5 (Circuit Hierarchy Theorem). *If $n \leq t(n) \leq 2^{n-2}/n$ then*

$$\text{Circuit}[t] \subsetneq \text{Circuit}[4t]$$

Proof. The inclusion is quite direct. To show proper inclusion we must exhibit a function on n variables that is not computable by circuits of size at most $t(n)$ but it is if allowed $4 \cdot t(n)$ gates.

Fix the maximal $m \in \{1, \dots, n\}$ s.t. $t(n) \leq 2^m/m \leq 2 \cdot t(n)$. This is possible since if m is the largest number such that $2^m/m \leq 2 \cdot t(n)$ then $2^{m+1}/(m+1) > 2 \cdot t(n) \implies t(n) \leq 2^m/(m+1) \leq 2^m/m$. Now consider the set of all boolean functions $B_{n,m}$ in $n > m$ variables that only depend on the first m bits (all the rest are fixed). By the Shannon-Lupanov lower bound, there exists a function $f_n \in B_{n,m}$. □

3 Det/nondet-erministic Communication Complexity

3.1 Deterministic Communication Complexity on 0-1 matrices

Definition 3.1. Consider the 0-1 matrix A , we consider the relation $F \subseteq X \times Y \times Z$ such that given $(x, y, z) \in F \iff A[x, y] = z$.

1. The deterministic communication complexity of matrix A (or equivalently F) is the minimum depth of a communication tree for A .
2. The tiling number $\chi(A)$ of a boolean matrix A is the samllest number of pairwise disjoint monochromatic submatrices of A covering all entries of A .

We have the following inequalities:

Theorem 3.1. $c(A) \geq \log(\chi(A))$ since there are $2^{c(A)}$ leaves in a communication tree which form a monochromatic covering and $\chi(A)$ is min over that. $c(A) \leq 2(\log(\chi(A)))^2$ by the bounds for every general relation F .

$\chi_0(A) + \chi_1(A) = \chi(A)$ and $\chi_0(A) = \chi_1(\bar{A})$. $\chi_1(A)$ is just the least number t such that A can be written as a sum of t matrices that have a all-1s submatrix and zero throughout.

$$A = \begin{pmatrix} 1 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 1 & 1 \end{pmatrix}$$

$$A = R_1 + R_2 + R_3, \chi_1(A) = 3$$

where

$$R_1 = \begin{pmatrix} 1 & 1 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}, \quad R_2 = \begin{pmatrix} 0 & 0 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}, \quad R_3 = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 1 & 1 \end{pmatrix}.$$

Each, R_i is of the form $u_i v_i^T$ with $u_i, v_i = \{0, 1\}^n$

Definition 3.2. (The many characterisations of the Real Rank) If the real rank $rk(A)$ of a boolean matrix A is $r > 0$, TFAE:

- r is the least number such that $A = \sum_i^r \alpha_i R_i$ for some 0-1 matrices R_i of the form $u_i v_i^T$ with $u_i, v_i = \{0, 1\}^n$.
- if A is symmetric then r is the number of nonzero eigenvalues of A . (Follows by diagonalizability)

Then, $c(A) \geq \log(\chi_1(A)) \geq \log(rk(A))$. $n \times n$ identity matrix has large $c(A)$ because it's full rank. Having ones on the diagonal is actually a bottleneck property.

Theorem 3.2. If A is a symmetric $n \times n$ boolean matrix with 1s on the diagonal, then

$$\chi(A) \geq \frac{n^2}{|A|}$$

where $|A|$ is the total number of ones in A .

Proof. Let $\lambda_1, \lambda_2, \dots, \lambda_n$ be the eigenvalues of A , at most $r = rk(A)$ many are non-zero. We know that $tr(A) = \sum_i^n \lambda_i$ and thus by Cauchy-Schwarz

$$tr(A^2) = \left(\sum_i^n \lambda_i^2 \right) = \left(\sum_{i, \lambda_i \neq 0}^n \lambda_i^2 \right) \geq tr^2(A)/r$$

Since A is a 0-1 matrix, $tr(A^2) = |A|$ (verify), since all ones are on the diagonal, the A_{ii}^2 entry counts the number of ones in the i th row of A . $\square$

3.2 Nondeterministic Communication Complexity

Definition 3.3. Cover number $Cov(A)$ of a 0-1 matrix A is the smallest number of all-1 submatrices of A covering all its 1s, this time the matrices in a cover need not be disjoint. Equivalently, $Cov(A)$ is the least positive integer t such that A can be written as a componentwise OR, $A = \bigvee_{i=1}^t R_i$ of t primitive matrices $R_1, \dots, R_t$, that is boolean matrices of rank 1.

A non-deterministic communication tree is where a third party having access to both Alice's and Bob's input x, y respectively provides a witness string $w \in \{0, 1\}^t$ convincing them that $A[x, y] = 1$. If this is true, then some witness string must exist and Alice and Bob must both say YES. If not, then at least one of them must say NO on every witness w .

Definition 3.4. X is the set of rows and Y is the set of columns of A then a non-deterministic protocol of complexity t is a pair of functions, $a : X \times \{0, 1\}^t \rightarrow \{0, 1\}$ and $b : \{0, 1\}^t \times Y \rightarrow \{0, 1\}$ such that

$$A[x, y] = 1 \iff \exists w \in \{0, 1\}^t \text{ s.t. } a(x, w) \wedge b(w, y) = 1$$

The nondeterministic communication complexity $\mathbf{nc}$ of a boolean matrix A is defined as the smallest number t on the size of the witness string for which the functions a, b exist verifying it. It is related in the following way to the covering number.

Lemma 3.3. For every boolean matrix A , we have that

$$\log Cov(A) \leq \mathbf{nc}(A) \leq \lceil \log Cov(A) \rceil$$

Proof. Consider the upper bound. Let $C := Cov(A)$ then there are all-1 submatrices (not necessarily disjoint) $R_1, \dots, R_C$. We define the functions a, b given witness $w \in \{0, 1\}^{\lceil \log_2(C) \rceil}$, such that $a(x, w) = 1 \iff x$ is a row of matrix indexed by w i.e. R_w . Similarly, $b(w, y) = 1 \iff y$ is a column of R_w . For the lower bound, for each witness $w \in \{0, 1\}^t$, the places where both functions output 1 form an all-1s submatrix. The union for all such witnesses for the YES cases form a cover. There are 2^t of those. $Cov(A) \leq 2^t$ thus $\log(Cov(A)) \leq t = \mathbf{nc}(A)$. $\square$

For all intents and purposes $nc(A) = \lg(Cov(A))$

Remark 3.4. Only sparse matrices have a high nondeterministic communication complexity. If every column or row contains of matrix A contains at most d zeros, then $Cov(A) = O(d \ln |A|)$.

3.3 Fooling Set Bounds

Remark 3.5. An all-(1) submatrix need not use consecutive rows or columns. For example, the (1)-entries of the row

$$[1, 0, 0, 1, 1, 1]$$

are covered by the single rectangle

$$r \times \{1, 4, 5, 6\}.$$

Thus any set of (1)-entries lying in one row, or one column, is contained in a single all-(1) rectangle.

Definition 3.5. Any 1-entries are independent if they do not lie in one row or in one column. If A is the adjacency matrix of a *bipartite* graph G :

- $\text{trk}(A)$ = maximum number of edges in a matching in G = largest number of pairwise independent 1-entries.
- $\omega(A)$ = maximum r such that G contains a complete $r \times r$ graph $K_{r,r}$ as a subgraph = largest $r \times r$ submatrix in G .
- $\lambda(A)$ = maximum degree of a vertex in G = largest number of 1s in a line (row or column)

Lemma 3.6.

$$Cov(A) \geq \frac{\text{trk}(A)}{\omega(A)} \geq \frac{|A|}{\lambda(A) \cdot \omega(A)}$$

Lemma 3.7. For every nonzero boolean matrix A ,

$$Cov(A) \geq \frac{\text{trk}(A)^2}{|A|}$$

Proof. Given a indices of I of independent entries with $|I| = \text{trk}(A)$ consider a cover of 1-entries in A by $R_1, \dots, R_t$ primitive matrices. Now consider mapping each $i = (x, y) \in I$ to the matrix R_j that first covers it.

□

3.4 P and $\text{NP} \cap \text{coNP}$ for Fixed Partition Games

For deterministic communication complexity, a tiling of A corresponds to a tiling of $\bar{A} = A - J$, where J is the $m \times n$ all-ones matrix.

How can we compare this to nondeterministic communication complexity? There, we only need to cover the 1s of the matrix A and the submatrices need not be disjoint. Moreover, covering $\bar{A}$ is not the same as covering A .

Theorem 3.8. (*Aho et.al 1983*) $c(A) \leq 2(\max\{nc(A), nc(\bar{A})\} + 1)^2$

Proof. Let A be $m \times n$. We cover the matrix first and then find a tiling. Consider $\mathcal{R} = \mathcal{R}_0 \cup \mathcal{R}_1$, where $|\mathcal{R}_0| \leq 2^{nc(\bar{A})}$ is a covering of the 0 entries by all-0 submatrices and $|\mathcal{R}_1| \leq 2^{nc(A)}$ is covering of 1s of A by all-1 matrices. The matrices in $\mathcal{R}_0, \mathcal{R}_1$ are disjoint, so to tile all of A we can turn this into a find-a-rectangle game.

Given an arbitrary relation $R = X \times Y \times Z$, Alice gets an $x \in X$, Bob gets a $y \in Y$ and their goal is to find a label of a combinatorial rectangle containing (x, y) . A legal labelling is such that if two combinatorial rectangles R, S have different labels then $R \cap S = \emptyset$.

Label matrices in $\mathcal{R}_0$ as "0" and those in $\mathcal{R}_1$ as "1". This labelling is legal. Finding a tiling is now equivalent to finding a label for this relation over the relation $R = [m] \times [n] \times \{"0", "1"\}$ for each point $(x, y) \in [m] \times [n]$.

Lemma 3.9. For an arbitrary relation $R = X \times Y \times Z$ we have that $c(R) \leq 2(\log |R|)^2$.

So $c(\mathcal{R}) \leq 2(\log |\mathcal{R}|)^2 = 2\max\{nc(A), nc(\bar{A})\}^2$. Let $K = \max\{nc(A), nc(\bar{A})\}$ then $\log(|\mathcal{R}|)^2 = \log(|\mathcal{R}_0| + |\mathcal{R}_1|)^2 \leq \log(2^{nc(A)} + 2^{nc(\bar{A})})^2 \leq (\log(2^K + 2^K))^2 = (K+1)^2$. $\square$

Remark 3.10. The k -disjointness matrix $D_{n,k}$ is a tight example for the above bound. This is a matrix whose rows and columns are indexed by subsets of $[n]^{(\leq k)}$, and

$$D_{n,k}[a, b] = \begin{cases} 0, & \text{if } a \cap b \neq \emptyset, \\ 1, & \text{if } a \cap b = \emptyset, \end{cases} \quad a, b \in [n]^{(\leq k)}.$$

For all $1 \leq k \leq n$ we have that

$$nc(\overline{D_{n,k}}) \leq \log n$$

$$nc(D_{n,k}) = O(k + \log \log n)$$

but

$$c(D_{n,k}) \geq \log \text{rank}(D_{n,k}) = \Omega(k \log(n/k))$$

3.5 Rank and Conjectures

(*Log-Rank Conjecture*) There is a constant $c > 0$ such that for every 0-1 matrix A ,

$$c(A) \leq (\log(\text{rank}(A)))^c$$

Over finite fields – instead of the reals, this conjecture does not hold.

Since we know that $c(A) \geq \log(\chi(A))$ and $\chi(A) \geq \frac{n^2}{\text{mono}(A)}$ where $\text{mono}(A)$ is the largest monochromatic submatrix of A . We reach the following conjecture.

(*Nisan-Widgerson Conjecture*) There is a constant $c > 0$ such that for every boolean $n \times n$ matrix A of rank r ,

$$\text{mono}(A) \geq \frac{n^2}{2^{(\log r)^c}}$$

Another reformulation of the Log-Rank conjecture concerns the chromatic number $\text{ch}(G)$ and the real rank of the adjacency matrix of G .

(*Chromatic Number Conjecture*) There is a constant $c > 0$ such that for every graph with adjacency matrix of rank r ,

$$\text{ch}(G) \leq 2^{(\log r)^c}$$

was shown equivalent to Log-Rank by Lovász and Saks.

3.6 Nisan-Widgerson Discrepancy Lemma

Definition 3.6. For A a ± 1 matrix that is $n \times n$,

$$\text{disc}(A) = \max |x^T A y| = \max \left| \sum_{i,j=1}^n a_{ij} x_i y_j \right|$$

where $x, y \in \{0, 1\}^n$, each pair of such vectors corresponds to choosing a submatrix of A determined by the 1-position of x and y . Obviously, $0 \leq \text{disc}(A) \leq n^2$ for every $n \times n$ matrix, where equality holds if A is monochromatic.

This replacing 'mono' with 'disc' (and Boolean matrices to ± 1 matrices) in the Nisan-Widgerson conjecture gives us a stronger version of the conjecture which is actually true.

Theorem 3.11. For every $n \times n$ ± 1 matrix A of rank r ,

$$\text{disc}(A) \geq \frac{n^2}{16r}$$

Theorem 3.12. *There exist explicitly given 0 – 1 matrices A of size $2^n \times 2^n$ such that $c(A) = \Omega(n)$ and $\log \text{rank}(A) = O(n^\alpha)$ where $\alpha = \log_3(2)$*

Proof. Define for every boolean function f on n vars,

$$A_f[x, y] := f(x_1 \cdot y_1, x_2 \cdot y_2, \dots, x_n \cdot y_n), \forall i, x_i, y_i \in \{0, 1\}^n$$

- If f is fully sensitive at the zero vector $\mathbf{0}$, that is, $f(\mathbf{0}) = 0$ and $f(x) = 1$ for all vectors with exactly one 1 then $c(A_f) = \Omega(n)$.

This follows from the fact that the following matrix has $\Omega(n)$ communication complexity.

$$UDISJ_n[x, y] = \begin{cases} 1 & \text{if } \sum_{i=1}^n x_i \cdot y_i = 0 \\ 0 & \text{if } \sum_{i=1}^n x_i \cdot y_i = 1 \end{cases}$$

If f is fully sensitive then the corresponding matrix A_f is of the above type.

- On the other hand, if polynomial of f , that is $f(z) = \sum_S \lambda_S \Pi_{i \in S} z_i$, has m monomials, then $\text{rank}(A_f) \leq m$.

By definition of A_f it follows $A_f = \sum_S \lambda_S B_S$ where $B_s[x, y] := \Pi_{i \in S} x_i \cdot y_i$ has rank 1. Thus by rank inequality, $\text{rank}(A) \leq \sum_S \text{rank}(B_S) \leq m \cdot 1$.

The above are properties satisfied by considering $E_3(x, y, z) = 1 \iff x + y + z \in \{1, 2\}$ which is equivalent to

$$E_3(x, y, z) = x + y + z - xy - xz - yz$$

as a degree-2 multilinear poly. Then define f_m on $n = 3^m$ variables obtained by building a complete ternary tree of depth m where 3^m leaves are distinct variables and each node is the E_3 fn of its three children. This function satisfies being fully sensitive and having $2^{O(n^\alpha)}$ monomials in polynomial form. $\square$

Let $A \in \mathbb{R}^{m \times n}$. The Frobenius norm is

$$\|A\|_F = \left(\sum_{i,j} A_{ij}^2 \right)^{1/2},$$

equivalently if A has rank r and singular values $\sigma_1 \geq \dots \sigma_r > 0$ then

$$\|A\|_F = \sum_{i=1}^r \sigma_i^2$$

The spectral norm is

$$\|A\|_{\text{spec}} = \sup_{\|x\|_2=1} \|Ax\|_2.$$

and equivalently,

$$\|A\|_{\text{spec}} = \sup_{\|x\|_2=\|y\|_2=1} |y^T Ax|.$$

equivalently, it is the largest singular value σ_1 .

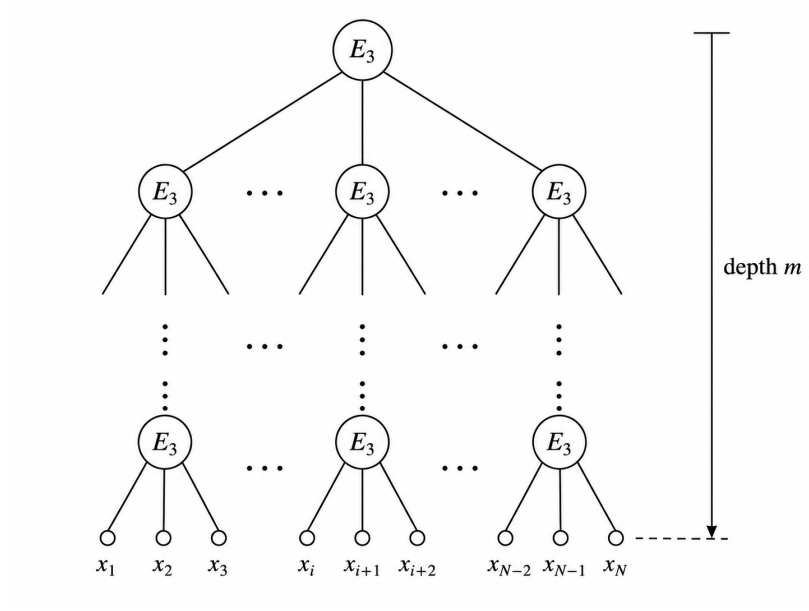


Figure 2: Define output $f_m(x_1, \dots, x_N)$ where $N = 3^m$ as the output of the top E_3 gate which is in $\{1, 2\}$

Lemma 3.13. $\frac{\|A\|_F}{\sqrt{\text{rank}(A)}} \leq \|A\|_{\text{spec}} \leq \|A\|_F$

Proof. Equivalently, if $\sigma_1 \geq \dots \geq \sigma_r > 0$ are the nonzero singular values of A , where $r = \text{rank}(A)$, then

$$\|A\|_{\text{spec}} = \sigma_1, \quad \|A\|_F^2 = \sum_{i=1}^r \sigma_i^2.$$

Hence

$$\sigma_1^2 \leq \sum_{i=1}^r \sigma_i^2 \leq r \sigma_1^2.$$

Taking square roots gives

$$\|A\|_{\text{spec}} \leq \|A\|_F \leq \sqrt{\text{rank}(A)} \|A\|_{\text{spec}},$$

or equivalently,

$$\frac{\|A\|_F}{\sqrt{\text{rank}(A)}} \leq \|A\|_{\text{spec}} \leq \|A\|_F.$$

□

4 Adjacency Functions, Graph Representations, and Star Complexity

Let $G \subseteq L \times R$ be a bipartite graph, where

$$L = \{0, 1\}^\ell, \quad R = \{0, 1\}^r.$$

There are three related but distinct ways to view the computation of G . First, the *adjacency function* of G is the Boolean function

$$f_G : \{0, 1\}^{\ell+r} \rightarrow \{0, 1\}$$

defined by

$$f_G(u, v) = 1 \iff (u, v) \in G.$$

Here the input consists of the binary encodings of two vertices:

$$u \in \{0, 1\}^\ell, \quad v \in \{0, 1\}^r.$$

Thus f_G is an ordinary Boolean function whose variables are the bits

$$y_1, \dots, y_\ell, z_1, \dots, z_r.$$

It computes whether the two encoded vertices are adjacent in G .

Second, one may represent G by a Boolean function over variables indexed by vertices. Introduce one variable for each vertex:

$$X = \{x_w : w \in L \cup R\}.$$

A Boolean function $F(X)$ *represents* G if, for every $u \in L$ and $v \in R$,

$$F(e_u + e_v) = 1 \iff (u, v) \in G.$$

Here $e_u + e_v$ is the vector with a 1 in the coordinate corresponding to u , a 1 in the coordinate corresponding to v , and 0's elsewhere. Thus F is not given the binary names of u and v . Instead, it is given the two-vertex set $\{u, v\}$. Importantly, F is only required to be correct on inputs with exactly two 1's; on all other inputs, its value is irrelevant. Third, one can compute G directly as a graph, starting from stars. For each vertex w , let

$$S_w = \{\text{edges incident to } w\}$$

be the star centered at w . A graph circuit builds graphs from these stars using only the operations

$$\cup \quad \text{and} \quad \cap.$$

The *star complexity* of G , denoted $\text{Star}(G)$, is the minimum number of union and intersection operations needed to obtain G from the stars.

The connection between the second and third viewpoints is the dictionary

$$x_w \longleftrightarrow S_w, \quad \vee \longleftrightarrow \cup, \quad \wedge \longleftrightarrow \cap.$$

Indeed, the variable x_w represents the star S_w , because

$$x_w(e_u + e_v) = 1 \iff w = u \text{ or } w = v,$$

which is exactly the condition that the edge (u, v) is incident to w . Moreover, if functions g and h represent graphs G_1 and G_2 , then

$$g \vee h$$

represents $G_1 \cup G_2$, while

$$g \wedge h$$

represents $G_1 \cap G_2$.

Therefore, computing G from stars using $\{\cup, \cap\}$ is exactly the same as representing G by a monotone Boolean circuit over the variables $\{x_w : w \in L \cup R\}$. Hence

$$\text{Star}(G) := \text{minimum size of a monotone circuit representing } G.$$

This should be distinguished from the ordinary circuit complexity of the adjacency function f_G . The function f_G takes bit-encodings of vertices as input, whereas a representing function $F(X)$ takes vertex-indicator vectors and only needs to behave correctly on two-vertex inputs.

Magnification Lemma. Let $G \subseteq L \times R$ be a bipartite graph with

$$L = \{0, 1\}^\ell, \quad R = \{0, 1\}^r,$$

and let

$$f_G : \{0, 1\}^{\ell+r} \rightarrow \{0, 1\}$$

be its adjacency function, defined by

$$f_G(u, v) = 1 \iff (u, v) \in G.$$

Suppose $C(y, z)$ is a Boolean circuit computing f_G , where the inputs are literals

$$y_i^a, z_j^a \quad (a \in \{0, 1\}).$$

Introduce one new variable x_w for every vertex $w \in L \cup R$. For each literal y_i^a , define

$$Y_i^a(X) = \bigvee_{\substack{w \in L \\ w(i)=a}} x_w,$$

and for each literal z_j^a , define

$$Z_j^a(X) = \bigvee_{\substack{w \in R \\ w(j)=a}} x_w.$$

Now replace every input literal of C as follows:

$$y_i^a \mapsto Y_i^a(X), \quad z_j^a \mapsto Z_j^a(X).$$

The resulting circuit $C'(X)$ is positive, i.e. monotone, and represents G : for every $u \in L$ and $v \in R$,

$$C'(e_u + e_v) = 1 \iff C(u, v) = 1 \iff f_G(u, v) = 1 \iff (u, v) \in G.$$

Thus, a circuit computing the adjacency function f_G can be transformed into a monotone circuit representing G . In circuit language, the lemma switches from a Boolean circuit on the bit-encodings of vertices to a monotone circuit on vertex-indicator variables.

Theorem 4.1 (Strong Magnification Lemma). *For circuits having fan-in 2 on AND, OR we define complexity measure $\text{Circuit}(f_G)$. For every bipartite $n \times m$ graph G ,*

$$\text{Circuit}(f_G) \geq \text{Star}(G) - 2(n + m) - 8(\sqrt{n} - \sqrt{m})$$

Proof Sketch. We are going to show that the replacements of the ORs we made in the magnification lemma are *simultaneously* computable in relatively low number of fanin-2 ORs.

To show this we look at the disjunction and define a matrix $A = (a_{ij})$ where $a_{ij} = 1 \iff j \in S_i$ so that we have,

$$\bigvee_{j \in S_i} x_j = \bigvee_{j: a_{ij}=1} x_j = \bigvee_{j=1}^n a_{ij} x_j$$

if we compute this matrix, we can simultaneously compute all disjunctions.

Let $D(A)$ be the complexity measure of fan-in 2 AND, OR circuits computing this matrix.

Lemma 4.2. (Transposition Principle) If A is a boolean matrix with m rows and n columns, then $D(A^T) = D(A) + m - n$.

Lemma 4.3. For every integer $1 \leq s \leq m$, every collection of m boolean sums of n variables can be simultaneously computed by a circuit consisting solely of at most $sn + s2^{m/s} - 2m - s$ fan-in 2 OR gates.

Proof. Given $m \times n$ matrix A , consider A^T . Split it into s submatrices of size $n \times k$ and $k \leq m/s$. Brute force disjunctions on k variables per submatrix which takes $2^k - k - 1$ OR gates. The combine using $n(s - 1)$ OR gates. $D(A) \leq D(A^T) + n - m \leq s(2^k - k - 1) + n(s - 1) \leq s2^{m/s} - m - s + n(s - 1) + n - m$ which gives bound. $\square$

Lemma 4.4. Simultaneously computing A and its complement $J - A = \bar{A}$ denoted by $D(A, \bar{A}) \leq q + 2^{p+2}$

In the proof of magnification lemma, the matrix A corresponds to the $l = \log_2(n)$ y -variables by a boolean disjunction of $n = |L|$ new variables. Let A be the $l \times n$ matrix (linear transformation) corresponding to this set of boolean sums. The sums corresponding to negations of ys is $\bar{A}$.

Further split A into two $(l/2) \times n$ submatrices A_1, A_2 then applying above lemma with $p = l/2 = \log_2 n/2$ and $q = n$, $D(A, \bar{A}) \leq D(A_1, \bar{A}_1) + D(A_2, \bar{A}_2) \leq 2(n + 4\sqrt{n})$. Do the same with the $r = \log_2 m$ z -variables being replaced by disjunction of $m = |R|$ variables and the total circuit must have size $2(n + 4\sqrt{n}) + 2(m + 4\sqrt{m})$. So given a circuit fanin 2 AND/OR computing f_G , we transform it by adding at most $2(n + 4\sqrt{n}) + 2(m + 4\sqrt{m})$ to get a circuit representing G which is precisely the star complexity. Therefore,

$$\text{Star}(G) \leq \text{Circuit}(f_G) + 2(n + m) + 8(\sqrt{n} + \sqrt{m})$$

□

Polynomial graph complexity lower bound gives exponential circuit lower bounds if are allowed unbounded fan-in in our circuits.

Monotone circuits have fan-in 2 AND and OR gates, can have unbounded fan-out and inputs are literals and their negations. Circuits with fanout 1 makes it a formula in addition to those conditions satisfied by monotone circuits, i.e. circuit is binary tree. Leafsize will mean number of leaves in its underlying binary tree.

Definition 4.1. $L(f)$ denotes the smallest leafsize of a formula comparing boolean function f . For a graph G , we let $L_+(G)$ denote the smallest leafsize of a monotone formula representing G . (Obviously these are two different things).

Magnification lemma gives us that $L(f_G) \geq \frac{2}{n} L_+(G)$. If $n = 2^l$, then the adjacency function f_G is a boolean function in $2l = 2 \log_2 n$ variables.